Session Type State Spaces Form Lattices

Alexandre Zua Caldeira
LASIGE, Faculdade de Ciências, Universidade de Lisboa, Portugal

Vasco T. Vasconcelos
LASIGE, Faculdade de Ciências, Universidade de Lisboa, Portugal

Abstract

We prove that the state space of every well-formed session type, quotiented by strongly connected components, forms a bounded lattice. The proof proceeds by structural induction on the session type: **end** gives a trivial lattice; sequencing extends by a chain element; branch and selection create the meets and joins required for pairs of states in distinct sub-state-spaces; recursion is handled via a top absorption lemma on the SCC quotient; and parallel composition yields a product lattice. We call the resulting lattice structure a *reticulate*. We validate this result with two independently developed tools — Reticulate (Python, 642 tests) and Bica Reborn (Java, 733 tests) — on 30 real-world protocol benchmarks drawn from networking, databases, distributed systems, and AI. All 30 are confirmed to form lattices. The two most technically involved lemmas have been mechanically verified in Lean 4 with Mathlib. This lattice structure provides a principled algebraic foundation for session type analysis and enables the parallel composition constructor ‖ through product lattice construction.

2012 ACM Subject Classification Theory of computation → Type theory; Theory of computation → Process calculi; Software and its engineering → Formal methods

Keywords and phrases session types, lattice theory, state spaces, parallel composition, structural induction

Digital Object Identifier 10.4230/LIPIcs...

Related Version Tool and benchmarks: `https://github.com/alcides/reticulate`

Supplementary Material Interactive demo: `https://bica.zuacaldeira.com`

Funding *Alexandre Zua Caldeira*: Funded by FCT (Fundação para a Ciência e a Tecnologia).

1 Introduction

Session types [14, 25] describe the communication protocols that govern interaction with objects, channels, or services. A session type specifies the allowed sequences of operations—method calls, message sends, branches, selections—together with their ordering constraints. The *state space* of a session type is the labeled transition system (LTS) obtained by "executing" the type: each state represents a protocol stage, and each transition a permitted action.

State spaces of session types are routinely used for verification, subtype checking, and protocol analysis, yet their algebraic structure has received surprisingly little attention. They are typically treated as automata or labeled transition systems, with algorithmic techniques (reachability, bisimulation, language inclusion) applied generically. In this paper we show something stronger:

> *The state space of every well-formed session type, after quotienting by strongly connected components, is a bounded lattice.*

We call this lattice a *reticulate*. The lattice structure arises naturally from the session type constructors. Branch (&) and selection (⊕) create fan-out points that serve as meets and joins for states in distinct sub-state-spaces. Sequencing (.) extends by a chain element.

© Alexandre Zua Caldeira and Vasco T. Vasconcelos;
licensed under Creative Commons License CC-BY 4.0

Recursion (μ) introduces cycles that are absorbed by the SCC quotient. And parallel composition ($\|$) yields the product of two lattices, which is itself a lattice with componentwise meets and joins.

The $\|$ constructor, introduced in our companion work [4], is where lattice structure becomes *necessary* rather than merely convenient. Without $\|$, a session type's state space is a tree, DAG, or simple diamond whose lattice structure is straightforward. With $\|$, the product of two state spaces inherently has meets and joins corresponding to the fork and synchronisation points of concurrent execution.

Contributions.

1. A formal, inductive construction $\mathcal{L}(S)$ that maps each session type S to its state-space LTS (Section 3).
2. A proof that $\mathcal{L}(S)/\equiv$ is a bounded lattice for every well-formed S, by structural induction (Section 4).
3. An extension to the parallel constructor $\|$ via product lattice construction (Section 4.4).
4. Empirical validation on 30 benchmark protocols drawn from diverse application domains (Section 6).
5. A mechanised proof of the key lemmas (Top Absorption and Recursion) in Lean 4 with Mathlib [20, 24] (Section 5).

Outline. Section 2 recalls background on session types and lattices. Section 3 defines the state-space construction. Section 4 presents the key lemmas and the main theorem. Section 5 describes our tool support. Section 6 evaluates the result on benchmarks. Section 7 discusses related work and Section 8 concludes.

2 Preliminaries

2.1 Session Types

We consider session types for objects [12, 26], describing the permitted sequences of method calls on a typed object. The syntax is:

$$
\begin{array}{rcll}
S & ::= & \&\{m_1 : S_1,\ \ldots,\ m_n : S_n\} & \text{(branch / external choice)} \\
 & \mid & \oplus\{l_1 : S_1,\ \ldots,\ l_n : S_n\} & \text{(selection / internal choice)} \\
 & \mid & S_1 \parallel S_2 & \text{(parallel composition)} \\
 & \mid & \mu X.\, S & \text{(recursion)} \\
 & \mid & X & \text{(type variable)} \\
 & \mid & \mathbf{end} & \text{(terminated)}
\end{array}
$$

We write $m\,.\,S$ as syntactic sugar for $\&\{m : S\}$ (single-method branch, i.e., sequencing).

Branch $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ offers the environment a choice among methods $m_1, \ldots, m_n$; *selection* $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$ lets the object choose among labels $l_1, \ldots, l_n$. *Parallel composition* $S_1 \parallel S_2$ allows two sub-protocols to execute concurrently on the same object. *Recursion* $\mu X.\, S$ binds variable X in S.

▶ **Example 1.** A file object: $\mathit{open}\,.\,\mu X.\,\&\{\mathit{read} : \oplus\{\mathit{data} : X,\ \mathit{eof} : \mathit{close}\,.\,\mathbf{end}\}\}$. After opening, the client repeatedly reads; each read either returns data (continuing the loop) or signals end-of-file (leading to close).

▶ **Example 2.** A bidirectional channel: $(\mu X.\,\&\{\mathit{send} : X,\ \mathit{done} : \mathbf{end}\}) \parallel (\mu Y.\,\oplus\{\mathit{msg} : Y,\ \mathit{fin} : \mathbf{end}\})$. Sending and receiving proceed concurrently.

$$\frac{}{\mathsf{wf}(\mathbf{end})}\ \textsc{WF-End} \qquad \frac{\mathsf{wf}(S)}{\mathsf{wf}(m\,.\,S)}\ \textsc{WF-Seq} \qquad \frac{\mathsf{wf}(S_i) \quad (\forall\, i)}{\mathsf{wf}(C\{a_1{:}S_1, \ldots, a_n{:}S_n\})}\ \textsc{WF-Choice}$$

$$\frac{\mathsf{wf}(S) \qquad S \neq X}{\mathsf{wf}(\mu X.\, S)}\ \textsc{WF-Rec}$$

$$\frac{\begin{array}{c}\mathsf{wf}(S_1) \qquad \mathsf{wf}(S_2) \qquad \mathsf{term}(S_1) \qquad \mathsf{term}(S_2) \\ \mathsf{fv}(S_1) \cap \mathsf{bv}(S_2) = \emptyset \qquad \mathsf{fv}(S_2) \cap \mathsf{bv}(S_1) = \emptyset \qquad \neg\mathsf{haspar}(S_1) \qquad \neg\mathsf{haspar}(S_2)\end{array}}{\mathsf{wf}(S_1 \parallel S_2)}\ \textsc{WF-Par}$$

Figure 1 Well-formedness rules. $C \in \{\&, \oplus\}$. $\mathsf{term}(S)$: every maximal path reaches **end**. fv, bv: free and bound recursion variables. $\mathsf{haspar}(S)$: S contains a nested $\parallel$.

▶ **Example 3.** An SMTP connection:

$$\mathit{connect}\,.\,\mathit{ehlo}\,.\,\mu X.\,\&\{\mathit{mail} : \mathit{rcpt}\,.\,\mathit{data}\,.\,\oplus\{\mathit{OK} : X,\ \mathit{ERR} : X\},\ \mathit{quit} : \mathbf{end}\}$$

After connecting and issuing EHLO, the client enters a loop where it can either send a message (mail, rcpt, data, with the server replying OK or ERR) and continue, or quit. This protocol has 7 states and 8 transitions.

Well-formedness.

Following [4], the well-formedness judgment $\mathsf{wf}(S)$ is defined by the rules in Figure 1. Every constructor propagates well-formedness structurally; the interesting case is WF-Par, which requires both branches to be *terminating* ($\mathsf{term}(S)$: every maximal path reaches **end**), variable-disjoint, and $\parallel$-free. These conditions ensure that parallel branches always synchronise at a common termination point. Termination is decidable: for $\mu X.\, S$, check that S has a path to **end** not passing through X.

2.2 Lattice Theory

A *partially ordered set* (poset) is a pair $(P, \leq)$ where $\leq$ is reflexive, antisymmetric, and transitive. Given $a, b \in P$, the *meet* (greatest lower bound) is $a \wedge b = \max\{c \mid c \leq a \text{ and } c \leq b\}$, and the *join* (least upper bound) is $a \vee b = \min\{c \mid a \leq c \text{ and } b \leq c\}$, when they exist.

▶ **Definition 4** (Bounded lattice). *A* bounded lattice *is a poset* $(L, \leq)$ *with a least element* $\bot$ *and a greatest element* $\top$ *such that every pair of elements has both a meet and a join.*

▶ **Definition 5** (Product lattice). *Given bounded lattices* $(L_1, \leq_1)$ *and* $(L_2, \leq_2)$*, their* product lattice *is* $L_1 \times L_2$ *with componentwise ordering:* $(a_1, a_2) \leq (b_1, b_2)$ *iff* $a_1 \leq_1 b_1$ *and* $a_2 \leq_2 b_2$*. Meets and joins are componentwise:* $(a_1, a_2) \wedge (b_1, b_2) = (a_1 \wedge_1 b_1,\ a_2 \wedge_2 b_2)$ *and* $(a_1, a_2) \vee (b_1, b_2) = (a_1 \vee_1 b_1,\ a_2 \vee_2 b_2)$*.*

A *strongly connected component* (SCC) of a directed graph is a maximal set of vertices that are mutually reachable. The *SCC quotient* collapses each SCC to a single node; the result is always a directed acyclic graph (DAG) [23].

3 State-Space Construction

We now define the state-space construction $\mathcal{L}(S)$ that maps a session type S to a labeled transition system.

▶ **Definition 6** (State space). *A* state space *is a tuple* $\mathcal{L}(S) = (Q, \Sigma, \delta, q_\top, q_\bot)$ *where* Q *is a finite set of states,* Σ *is a finite alphabet of action labels (method names and selection labels),* $\delta \subseteq Q \times \Sigma \times Q$ *is the transition relation,* $q_\top \in Q$ *is the* initial state*, and* $q_\bot \in Q$ *is the* terminal state*.*

The construction is defined inductively on the structure of S. We give an overview here; the full formal definition appears in [4].

end. $\mathcal{L}(\mathbf{end}) = (\{q\}, \emptyset, \emptyset, q, q)$.
A single state, both initial and terminal.

Sequencing $m\,.\,S$**.** Let $\mathcal{L}(S) = (Q, \Sigma, \delta, q_\top^S, q_\bot^S)$. Add a fresh state q_0 and transition $q_0 \xrightarrow{m} q_\top^S$. Result: $(Q \cup \{q_0\}, \Sigma \cup \{m\}, \delta \cup \{(q_0, m, q_\top^S)\}, q_0, q_\bot^S)$.

Branch $\&\{m_1 : S_1, \ldots, m_n : S_n\}$**.** Build each $\mathcal{L}(S_i)$ with shared terminal state $q_\bot$. Add a fresh initial state q_0 with transitions $q_0 \xrightarrow{m_i} q_\top^{S_i}$ for $i = 1, \ldots, n$. The sub-state-spaces are otherwise disjoint.

Selection $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$**.** Identical construction to branch, with labels l_i instead of methods m_i.

Recursion $\mu X.\,S$**.** Allocate a placeholder state for X. Build $\mathcal{L}(S)$, replacing each occurrence of X with a back-edge to the placeholder. Merge the placeholder with the initial state of $\mathcal{L}(S)$. This creates cycles in the transition graph.

Parallel $S_1 \parallel S_2$**.** See Definition 10 below.

▶ **Definition 7** (Reachability ordering). *Given* $\mathcal{L}(S) = (Q, \Sigma, \delta, q_\top, q_\bot)$*, define* $s_1 \sqsupseteq s_2$ *iff there exists a (possibly empty) path from* s_1 *to* s_2 *in* (Q, δ)*.*

The relation $\sqsupseteq$ is a preorder (reflexive and transitive). It need not be antisymmetric due to cycles introduced by recursion.

▶ **Proposition 8** (Unique extrema). *For every well-formed session type* S*, the state space* $\mathcal{L}(S)$ *has:*

1. *a unique initial state* $q_\top$ *such that* $q_\top \sqsupseteq s$ *for all* $s \in Q$*; and*
2. *a unique terminal state* $q_\bot$ *such that* $s \sqsupseteq q_\bot$ *for all* $s \in Q$*.*

Proof. By induction on S. For **end**, the single state is both initial and terminal. For sequencing $m\,.\,S'$, the fresh state q_0 reaches $q_\top^{S'}$ via m, and $q_\top^{S'}$ reaches all states of $\mathcal{L}(S')$ by hypothesis; the terminal is inherited. For branch and selection, the fresh initial q_0 reaches each $q_\top^{S_i}$ directly, and the shared terminal $q_\bot$ is reachable from all states by the induction hypothesis on each S_i. For recursion $\mu X.\,S'$, the initial state is that of the body, which already reaches everything; back-edges only add reachability to $q_\top^{S'}$, preserving the property. For parallel $S_1 \parallel S_2$, the initial state is $(q_\top^1, q_\top^2)$, which reaches every (s_1, s_2) by interleaving a path from $q_\top^1$ to s_1 and a path from $q_\top^2$ to s_2. The terminal $(q_\bot^1, q_\bot^2)$ is reachable from every (s_1, s_2) by the same interleaving argument; well-formedness (termination) ensures that both $q_\bot^1$ and $q_\bot^2$ are reachable. ◀

▶ **Definition 9** (SCC quotient). *Let* $\equiv$ *be the equivalence relation on* Q *defined by* $s_1 \equiv s_2$ *iff* $s_1 \sqsupseteq s_2$ *and* $s_2 \sqsupseteq s_1$*. The* SCC quotient $\mathcal{L}(S)/{\equiv}$ *has states* $Q/{\equiv}$*, with* $[s_1] \sqsupseteq [s_2]$ *iff* $s_1 \sqsupseteq s_2$ *in the original. The quotient is a partial order (antisymmetric).*

Cycles arise only from recursion ($\mu X.\, S$ creates back-edges). The SCC quotient collapses each cycle into a single equivalence class, yielding a DAG. By Proposition 8, the quotient has a unique maximum $[q_\top]$ and a unique minimum $[q_\bot]$, making it a bounded poset.

▶ **Definition 10** (Product construction). *Given* $\mathcal{L}(S_1) = (Q_1, \Sigma_1, \delta_1, q^1_\top, q^1_\bot)$ *and* $\mathcal{L}(S_2) = (Q_2, \Sigma_2, \delta_2, q^2_\top, q^2_\bot)$, *define*

$$\mathcal{L}(S_1 \parallel S_2) = (Q_1 \times Q_2,\ \Sigma_1 \cup \Sigma_2,\ \delta_\times,\ (q^1_\top, q^2_\top),\ (q^1_\bot, q^2_\bot))$$

where $\delta_\times$ *contains:*

- $(s_1, s_2) \xrightarrow{a} (s'_1, s_2)$ *whenever* $s_1 \xrightarrow{a} s'_1$ *in* δ_1, *and*
- $(s_1, s_2) \xrightarrow{a} (s_1, s'_2)$ *whenever* $s_2 \xrightarrow{a} s'_2$ *in* δ_2.

The ordering is componentwise: $(s_1, s_2) \sqsupseteq (s'_1, s'_2)$ *iff* $s_1 \sqsupseteq_1 s'_1$ *and* $s_2 \sqsupseteq_2 s'_2$.

▶ Remark 11 (Product reachability). The componentwise characterisation holds because: ($\Rightarrow$) a path in the product projects to paths in each component (each transition moves exactly one component); ($\Leftarrow$) given paths $s_1 \sqsupseteq_1 s'_1$ and $s_2 \sqsupseteq_2 s'_2$, interleaving the two sequences of transitions in any order produces a path $(s_1, s_2) \sqsupseteq (s'_1, s'_2)$ in the product.

3.1 Worked Example

We illustrate the construction on the session type from Example 1: $S_{file} = open\,.\,\mu X.\,\&\{read : \oplus\{data : X,\ eof : close\,.\,\mathbf{end}\}\}$.

1. $\mathcal{L}(\mathbf{end}) = (\{q_5\}, \emptyset, \emptyset, q_5, q_5)$.
2. $\mathcal{L}(close\,.\,\mathbf{end})$: add fresh state q_4 with $q_4 \xrightarrow{close} q_5$. States: $\{q_4, q_5\}$, initial q_4, terminal q_5.
3. $\mathcal{L}(\oplus\{data : X,\ eof : close\,.\,\mathbf{end}\})$: add fresh state q_2 with $q_2 \xrightarrow{data} q_X$ (placeholder for X) and $q_2 \xrightarrow{eof} q_4$.
4. $\mathcal{L}(\&\{read : \ldots\})$: add fresh state q_1 with $q_1 \xrightarrow{read} q_2$. (Single-method branch is just sequencing.)
5. $\mathcal{L}(\mu X.\ \ldots)$: merge q_X with q_1 (the initial state of the body). The transition $q_2 \xrightarrow{data} q_X$ becomes $q_2 \xrightarrow{data} q_1$, creating a cycle.
6. $\mathcal{L}(open\,.\ldots)$: add fresh state q_0 with $q_0 \xrightarrow{open} q_1$.

The result has 5 states, 5 transitions, and 4 SCCs after quotient (the cycle $q_1 \to q_2 \to q_1$ collapses to one SCC). The quotient is a 4-element chain: $[q_5] < [q_4] < [q_1, q_2] < [q_0]$, which is trivially a bounded lattice.

4 The Lattice Theorem

We now prove the main result: the SCC quotient of every well-formed session type's state space is a bounded lattice. The proof proceeds by structural induction on S, with one lemma per constructor.

Throughout this section, we use the reachability ordering $\sqsupseteq$ from Definition 7, with $q_\top$ as the maximum and $q_\bot$ as the minimum. Since the SCC quotient is a partial order (anti-symmetric), we work with $\mathcal{L}(S)/\equiv$ and write $\leq$ for the induced partial order on equivalence classes, dropping the brackets when no confusion arises.

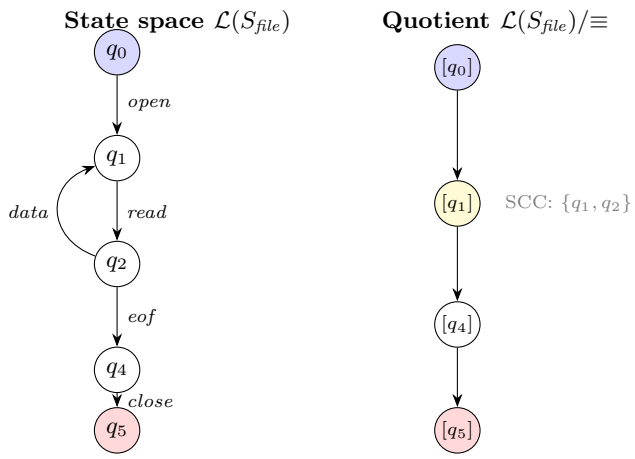

Figure 2 State space of the file object (Example 1) and its SCC quotient. The cycle $q_1 \to q_2 \to q_1$ (via *read* and *data*) collapses into a single SCC (highlighted). The quotient is a 4-element chain, trivially a bounded lattice.

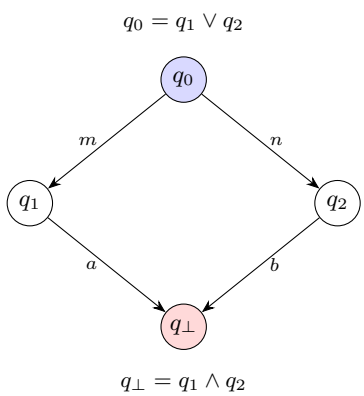

Figure 3 State space of $\&\{m : a \,.\, \mathbf{end},\ n : b \,.\, \mathbf{end}\}$, drawn as a Hasse diagram with $q_\top = q_0$ at the top and $q_\perp$ at the bottom. This is the diamond lattice M_2: the initial state is the join of q_1 and q_2, and the terminal state is their meet.

4.1 Base Cases

▶ **Lemma 12** (End). *$\mathcal{L}(\mathbf{end})/\equiv$ is a bounded lattice.*

Proof. The state space is a single state, which is both $\perp$ and $\top$. A one-element set is trivially a bounded lattice. ◀

▶ **Lemma 13** (Sequencing). *If $\mathcal{L}(S)/\equiv$ is a bounded lattice, then $\mathcal{L}(m \,.\, S)/\equiv$ is a bounded lattice.*

Proof. The construction adds a fresh state q_0 above the initial state of $\mathcal{L}(S)$, with a single transition $q_0 \xrightarrow{m} q_\top^S$. Since q_0 is fresh, no existing state has a transition to q_0, so q_0 has no incoming edges and is strictly above all states of $\mathcal{L}(S)$. In the quotient, $[q_0]$ is a new maximum element. Adding a new maximum to a bounded lattice yields a bounded lattice: for any s, t in the original lattice, their meet and join are unchanged; for $[q_0]$ and any $[s]$, we have $[q_0] \vee [s] = [q_0]$ and $[q_0] \wedge [s] = [s]$. ◀

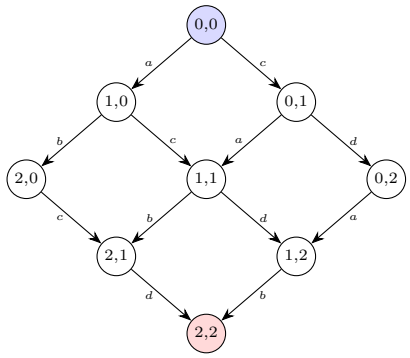

Figure 4 Product lattice $\mathcal{L}(a\,.\,b\,.\,\mathbf{end} \parallel c\,.\,d\,.\,\mathbf{end})$. The 9 states form a 3×3 grid lattice. State (i, j) denotes the product of the i-th state in the left component and j-th in the right ($0 =$ initial, $2 = \mathbf{end}$).

4.2 Choice Constructors

Branch and selection have identical state-space structure (both create a fan-out from a fresh initial state), so we treat them uniformly. We write $C\{a_1 : S_1, \ldots, a_n : S_n\}$ for either $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ or $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$.

▶ **Lemma 14** (Choice). *Let $S = C\{a_1 : S_1, \ldots, a_n : S_n\}$ with $n \geq 1$. If $\mathcal{L}(S_i)/{\equiv}$ is a bounded lattice for each $i = 1, \ldots, n$, then $\mathcal{L}(S)/{\equiv}$ is a bounded lattice.*

Proof. The construction creates:

- a fresh initial state q_0;
- for each i, a transition $q_0 \xrightarrow{a_i} q_\top^{S_i}$;
- the sub-state-spaces $\mathcal{L}(S_i)$, which are pairwise disjoint except for a shared terminal state $q_\bot$.

In the quotient $\mathcal{L}(S)/{\equiv}$, the maximum is $[q_0]$ and the minimum is $[q_\bot]$. We must show that every pair of states has a meet and a join. There are three cases.

Case 1: both states in the same sub-lattice. If $[s], [t] \in \mathcal{L}(S_i)/{\equiv}$, then their meet and join exist by the induction hypothesis on S_i. These are preserved in $\mathcal{L}(S)/{\equiv}$, since the embedding of $\mathcal{L}(S_i)/{\equiv}$ into $\mathcal{L}(S)/{\equiv}$ is order-preserving and order-reflecting (the sub-state-space is a convex sublattice between $[q_\top^{S_i}]$ and $[q_\bot]$).

Case 2: states in different sub-lattices. Let $[s] \in \mathcal{L}(S_i)/{\equiv}$ and $[t] \in \mathcal{L}(S_j)/{\equiv}$ with $i \neq j$.

We first establish *disjointness*: by construction, the state sets of $\mathcal{L}(S_i)$ and $\mathcal{L}(S_j)$ are disjoint except for the shared terminal state $q_\bot$, and the only transitions entering $\mathcal{L}(S_i)$ from outside are $q_0 \xrightarrow{a_i} q_\top^{S_i}$. In particular, no state in $\mathcal{L}(S_j) \setminus \{q_\bot\}$ has a path to any state in $\mathcal{L}(S_i) \setminus \{q_\bot\}$ that avoids q_0.

Meet. A common lower bound of $[s]$ and $[t]$ must be reachable from both. Since $[s]$ is internal to $\mathcal{L}(S_i)$ and $[t]$ to $\mathcal{L}(S_j)$, and these are disjoint above $[q_\bot]$, the only state reachable from both is $[q_\bot]$ (both sub-state-spaces reach $[q_\bot]$). Hence $[s] \wedge [t] = [q_\bot]$.

Join. A common upper bound of $[s]$ and $[t]$ must reach both. Since $[s] \in \mathcal{L}(S_i)$ is reachable only from states in $\mathcal{L}(S_i)$ or $[q_0]$, and similarly for $[t]$, the only state that reaches both is $[q_0]$. Hence $[s] \vee [t] = [q_0]$.

Case 3: one state is $[q_0]$ or $[q_\bot]$. Since $[q_0] = \top$ and $[q_\bot] = \bot$, meets and joins with these elements are trivial: $[q_0] \vee [s] = [q_0]$, $[q_0] \wedge [s] = [s]$, $[q_\bot] \vee [s] = [s]$, $[q_\bot] \wedge [s] = [q_\bot]$. ◀

▶ Remark 15 (Type-theoretic interpretation). Although branch and selection produce structurally identical state spaces, they have a well-known type-theoretic duality [11]. In the *subtyping* lattice, branch corresponds to a meet (intersection of offered behaviors) and selection to a join (union of chosen behaviors). Our result complements this: the *state space* lattice structure is uniform across both constructors, with the initial state serving as the join and the terminal state as the meet of cross-branch state pairs.

4.3 Recursion

The recursion case is the most delicate, since back-edges create cycles that cause the SCC quotient to merge classes. We first establish a general tool for reasoning about such merges.

▶ **Lemma 16** (Top absorption). *Let $(L, \leq)$ be a bounded lattice with greatest element $\top$, and let $D \subseteq L$ be an* upward-closed *set containing $\top$ (i.e., if $d \in D$ and $d \leq x$, then $x \in D$). Define $L' = (L \setminus D) \cup \{\top\}$ with the restricted ordering. Then L' is a bounded lattice.*

Proof. Since D is upward-closed and $L \setminus D$ is downward-closed (an order-ideal), the bottom element $\bot \notin D$ (otherwise $D = L$, contradicting that L' has at least $\top$ and $\bot$). So L' has least element $\bot$ and greatest element $\top$.

Let $a, b \in L'$. If either is $\top$, meets and joins are trivial. Otherwise $a, b \in L \setminus D$, and their meet $m = a \wedge_L b$ and join $j = a \vee_L b$ exist in L.

Meet. Since $m \leq a$ and $m \leq b$, and $L \setminus D$ is downward-closed, $m \in L \setminus D \subseteq L'$. Any lower bound of a, b in L' is also a lower bound in L, so m remains the greatest lower bound. Hence $a \wedge_{L'} b = m$.

Join. If $j \in L \setminus D$, then $j \in L'$ and the same argument as for meets shows j remains the least upper bound. If $j \in D$, then every element of D that is above both a and b has been collapsed into $\top$ in L'. We claim $a \vee_{L'} b = \top$. Indeed, $a \leq \top$ and $b \leq \top$. Any $c \in L'$ with $a \leq c$ and $b \leq c$ satisfies $j \leq c$ in L (since j is the join in L); if $c \neq \top$, then $c \in L \setminus D$, but $j \leq c$ and $j \in D$ contradicts $L \setminus D$ being downward-closed. So $c = \top$, confirming $\top$ is the least upper bound. ◀

▶ **Lemma 17** (Recursion). *If $\mathcal{L}(S)/{\equiv}$ is a bounded lattice (where S may contain free occurrences of X), then $\mathcal{L}(\mu X.\, S)/{\equiv}$ is a bounded lattice.*

Proof. The construction of $\mathcal{L}(\mu X.\, S)$ builds $\mathcal{L}(S)$ and adds back-edges from each occurrence of X to the initial state $q_\top^S$, creating cycles. We show that the SCC quotient of the resulting graph is a bounded lattice by reducing it to Lemma 16.

Step 1: Structural property of back-edges. Each free occurrence of X in S corresponds to a state s_X in $\mathcal{L}(S)$, and the recursion construction adds a transition $s_X \to q_\top^S$. Since $q_\top^S \sqsupseteq s_X$ already holds (the initial state reaches everything by Proposition 8), the new edge $s_X \to q_\top^S$ makes s_X and $q_\top^S$ mutually reachable, hence equivalent. More generally, every state on any path from $q_\top^S$ to s_X in $\mathcal{L}(S)$ becomes mutually reachable with $q_\top^S$.

Step 2: All merges collapse into $[q_\top^S]$. Let $D = \{[s] \in \mathcal{L}(S)/{\equiv_S} \mid s \equiv q_\top^S \text{ in } \mathcal{L}(\mu X.\, S)\}$ be the set of classes in the body's quotient that get absorbed into the initial state's class after adding back-edges. By Step 1, D contains $[q_\top^S]$ (the maximum of the body's lattice). Moreover, D is upward-closed: if $[s] \in D$ and $[s] \leq [r]$ in $\mathcal{L}(S)/{\equiv_S}$, then $q_\top^S \sqsupseteq r$ and $r \sqsupseteq s \sqsupseteq q_\top^S$ (the last step via some back-edge path), so $r \equiv q_\top^S$ in the recursive type, hence $[r] \in D$.

Step 3: Apply absorption. The quotient $\mathcal{L}(\mu X.\, S)/{\equiv}$ is isomorphic to the lattice obtained from $\mathcal{L}(S)/{\equiv_S}$ by collapsing the upward-closed set D into a single top element. By Lemma 16, the result is a bounded lattice. ◀

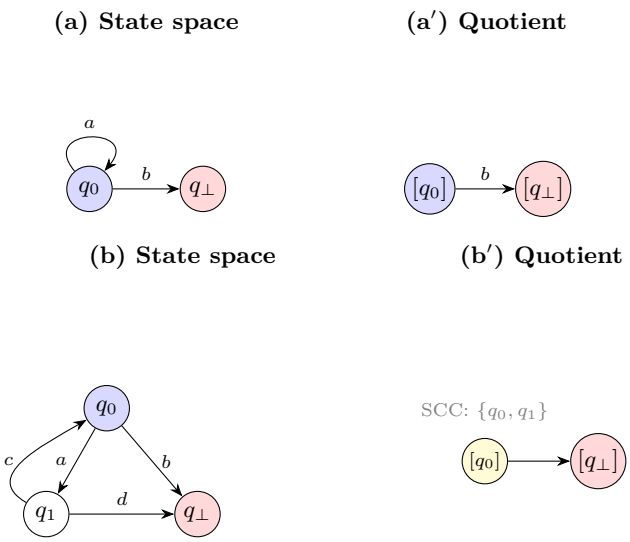

Figure 5 SCC quotients for recursive types. (a) $\mu X.\&\{a : X,\ b : \mathbf{end}\}$: the self-loop does not create a non-trivial SCC; quotient is a two-element chain. (b) $\mu X.\&\{a : \&\{c : X,\ d : \mathbf{end}\},\ b : \mathbf{end}\}$: the cycle $q_0 \to q_1 \to q_0$ collapses into one SCC; quotient is again a two-element chain. In both cases, the quotient is trivially a bounded lattice.

▶ **Example 18** (Recursion proof in action)**.** Consider $\mu X.\&\{a : \&\{c : X,\ d : \mathbf{end}\},\ b : \mathbf{end}\}$ (Figure 5b). The body $S = \&\{a : \&\{c : X,\ d : \mathbf{end}\},\ b : \mathbf{end}\}$ has state space with three states $q_0, q_1, q_\perp$ and quotient $\mathcal{L}(S)/{\equiv_S}$ equal to the three-element chain $[q_\perp] < [q_1] < [q_0]$, which is a bounded lattice.

The recursion adds a back-edge $q_1 \xrightarrow{c} q_0$, creating the cycle $q_0 \to q_1 \to q_0$. In the notation of Lemma 17: the absorbed set is $D = \{[q_0], [q_1]\}$. This is upward-closed: $[q_0] \in D$ (the maximum) and $[q_1] \in D$ (since $q_1 \to q_0$ makes $q_1 \equiv q_0$).

By Lemma 16, the quotient $\mathcal{L}(\mu X.\, S)/{\equiv} \cong \{[q_\perp], \top\}$ is a two-element chain—trivially a bounded lattice.

4.4 Parallel Composition

▶ **Lemma 19** (Parallel)**.** *If $\mathcal{L}(S_1)/{\equiv}$ and $\mathcal{L}(S_2)/{\equiv}$ are bounded lattices, then $\mathcal{L}(S_1 \parallel S_2)/{\equiv}$ is a bounded lattice.*

Proof. By Definition 10, $\mathcal{L}(S_1 \parallel S_2) = \mathcal{L}(S_1) \times \mathcal{L}(S_2)$ with componentwise ordering. By Remark 11, reachability in the product is componentwise. It follows that two product states (s_1, s_2) and (s_1', s_2') are in the same SCC iff $s_1 \equiv_1 s_1'$ and $s_2 \equiv_2 s_2'$. Hence

$$\mathcal{L}(S_1 \parallel S_2)/{\equiv} \;\cong\; (\mathcal{L}(S_1)/{\equiv_1}) \times (\mathcal{L}(S_2)/{\equiv_2}).$$

By Definition 5, the product of two bounded lattices is a bounded lattice, with componentwise meets and joins. ◀

4.5 Main Theorem

▶ **Theorem 20** (Reticulate Theorem)**.** *For every well-formed session type S, the SCC quotient $\mathcal{L}(S)/{\equiv}$ is a bounded lattice.*

Proof. By structural induction on S.

$S = \mathbf{end}$. Lemma 12: single-element lattice.

$S = m \,.\, S'$. By induction, $\mathcal{L}(S')/{\equiv}$ is a bounded lattice. Lemma 13: adding a new maximum preserves the property.

$S = \&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$ or $\oplus\{l_1{:}S_1, \ldots, l_n{:}S_n\}$. By induction, each $\mathcal{L}(S_i)/{\equiv}$ is a bounded lattice. Lemma 14: the choice constructor preserves the property.

$S = \mu X.\, S'$. By induction on the body S', $\mathcal{L}(S')/{\equiv}$ is a bounded lattice. Lemma 17: the SCC quotient of the recursive type is a bounded lattice.

$S = S_1 \parallel S_2$. By induction, $\mathcal{L}(S_1)/{\equiv}$ and $\mathcal{L}(S_2)/{\equiv}$ are bounded lattices. Lemma 19: the product is a bounded lattice. ◀

▶ **Corollary 21.** *The well-formedness condition* WF-Par *(termination of parallel branches, no cross-branch variables, no nested $\parallel$) is sufficient for lattice structure.*

Proof. WF-Par ensures that parallel branches are individually well-formed and terminating. By Theorem 20, each branch's state space is a bounded lattice, and their product is a bounded lattice by Lemma 19. ◀

4.6 Counterexamples and Necessity

The well-formedness conditions are sufficient but not all *necessary* for lattice structure. We examine what happens when they are violated.

▶ **Example 22** (Non-terminating recursion)**.** The type $\mu X.\, \&\{a : X\}$ is not well-formed (not terminating), but its state space has a single state (after SCC quotient), which is trivially a lattice. However, the type represents a protocol that can never complete—a degenerate case.

▶ **Example 23** (Non-terminating parallel)**.** The type $(\mu X.\, \&\{a : X\}) \parallel (\mu Y.\, \&\{b : Y\})$ violates WF-Par (neither branch terminates). Its product state space is $\{(q_0, q_0)\}$ with two self-loops, which is a one-element lattice. While technically a lattice, the protocol never reaches a terminal state, making it useless in practice.

▶ **Example 24** (Non-lattice from shared structure)**.** Consider a hypothetical state space (not arising from any session type) shaped like the "N_5" pentagon lattice with an extra edge removed. Such structures can fail to have pairwise meets or joins. The key insight of Theorem 20 is that the constructors of session types *never produce such structures*: the fan-out from choice and the product from parallel always yield well-behaved lattices.

▶ Remark 25 (Necessity). The question of which conditions are *necessary* for lattice structure remains open. Guardedness is needed to ensure the state space is well-defined (unguarded recursion $\mu X.\, X$ produces no transitions). Termination is needed for the terminal state $q_\perp$ to be reachable (the minimum of the lattice). The prohibition on nested $\parallel$ and cross-branch variables simplifies the product construction but may not be strictly necessary for lattice structure—relaxing these conditions is a direction for future work.

5 Tool Support

We have implemented the state-space construction, lattice checking, and related analyses in two independently developed tools, providing cross-validation of correctness.

Reticulate (Python).

A Python 3.12 library and command-line tool. It provides: a recursive-descent parser for the session type grammar of Section 2.1; state-space construction implementing the inductive rules of Section 3; product construction for the parallel constructor $\|$; SCC computation via an iterative variant of Tarjan's algorithm [23]; lattice checking by pairwise meet/join verification on the quotient DAG; termination checking and WF-Par validation; Hasse diagram visualisation via DOT/Graphviz output; and a CLI entry point supporting text, DOT, and rendered image output. The library comprises 8 modules with 642 unit tests, achieves full branch coverage on the core algorithms, and depends only on the Python standard library (Graphviz is an optional dependency for rendering).

Usage example:

```
$ python -m reticulate "rec X.&{a:X, b:end}"
Session type: rec X . &{a: X, b: end}
States: 2  |  Transitions: 2  |  SCCs: 2
Lattice check: IS a lattice
```

Bica Reborn (Java).

A Java 21 tool, successor to the original Bica [12]. It provides the same analysis pipeline (parse, state space, lattice check) plus: a Java annotation processor that can check session type conformance of Java source code at compile time, using annotations `@Session`, `@Shared`, `@ReadOnly`, and `@Exclusive`; a morphism hierarchy between state spaces (isomorphism, embedding, projection, Galois connection) [4]; a concurrency lattice for thread-safety analysis; and a standalone CLI tool. The tool comprises 10 packages with 733 unit tests. It uses Java sealed interfaces, records, and pattern matching for a type-safe AST representation.

Algorithm.

Both tools implement the same five-step algorithm:

1. **Parse.** Convert the session type string into an AST using recursive descent. The parser supports both ASCII (`&`, `+`, `||`) and Unicode ($\&$, $\oplus$, $\|$) notation.
2. **Build $\mathcal{L}(S)$.** Construct the state-space LTS by the inductive rules of Section 3. Recursion uses a placeholder-and-merge strategy: allocate a placeholder state for the variable, build the body, then redirect edges targeting the placeholder to the body's initial state.
3. **Compute SCCs.** Apply Tarjan's algorithm [23] (iterative variant, $\mathcal{O}(|Q| + |\delta|)$) to find strongly connected components.
4. **Quotient and reachability.** Form the quotient DAG and compute the reachability relation via topological-sort dynamic programming ($\mathcal{O}(|Q/{\equiv}|^2)$).
5. **Check meets and joins.** For each pair of SCC classes, verify that a greatest lower bound (meet) and least upper bound (join) exist in the quotient.

The worst-case complexity of Step 5 is $\mathcal{O}(|Q/{\equiv}|^3)$: for each of the $\mathcal{O}(|Q/{\equiv}|^2)$ pairs, finding the meet/join requires scanning $\mathcal{O}(|Q/{\equiv}|)$ candidates. For the protocols we consider (quotient sizes up to 20), this completes in under a millisecond.

Cross-validation.

The two tools were developed independently in different programming languages, yet agree on all 30 benchmarks: same state counts, transition counts, SCC counts, and lattice verdicts.

This provides strong evidence that both implementations correctly realise the constructions of Section 3.

Lean 4 formalisation.

The two most technically involved proof steps—the Top Absorption Lemma (Lemma 16) and the Recursion Lemma (Lemma 17)—have been mechanically verified in Lean 4 [20] using the Mathlib library [24]. The formalisation defines finite directed graphs, reachability, SCC quotients, and the quotient type L' from Lemma 16 as an inductive with constructors bot' (the collapsed set) and surv x (a survivor). The proof of Lemma 17 constructs an order-preserving bijection between the SCC quotient of the recursive graph and L', then transfers the lattice structure. The entire development compiles without `sorry` (Lean's axiom for unfinished proofs) and totals approximately 480 lines of Lean code.

6 Evaluation

We evaluate Theorem 20 empirically on 30 benchmark protocols drawn from diverse application domains: networking (SMTP, HTTP, DNS, TLS, MQTT, WebSocket), databases (JDBC transaction, connection pool), distributed systems (two-phase commit, Raft, Saga, leader replication, circuit breaker, failover), security (OAuth 2.0), AI agents (MCP [2], A2A [13]), programming patterns (Java Iterator, file object, reentrant lock, rate limiter, pub/sub), and tool pipelines (our own Reticulate pipeline, GitHub CI workflow).

Table 1 summarises the results. All 30 protocols are confirmed to form bounded lattices after SCC quotient, consistent with Theorem 20. Both tools agree on all metrics.

Key observations.

Of the 30 benchmarks, 12 (40%) use the parallel constructor. The largest is the Saga Orchestrator (20 states, 37 transitions), which combines parallel composition with sequencing and selection. Its product construction creates a $4 \times 4 = 16$ state intermediate that, after sequencing, grows to 20 states—all forming a lattice.

The Raft Leader Election protocol illustrates nested recursion: an outer loop for election rounds and an inner loop for the leader's append-entries cycle. After SCC quotient, the 6 original states collapse to just 2 SCCs (the two recursive loops), yielding a two-element chain.

The MCP protocol (7 states, 17 transitions) is notable for combining parallel composition with recursion in both branches, producing 5 SCCs in the quotient. Despite the complex interaction pattern (tool calls interleaved with notifications), the lattice structure emerges naturally.

All benchmarks complete in under 100 milliseconds on commodity hardware (both tools). The cubic meet/join check on the quotient dominates, but with quotient sizes ranging from 2 to 20, this is negligible in practice.

Lattice shape classification.

The 30 benchmarks exhibit several distinct lattice shapes after SCC quotient:

- *Chains* (8 protocols): Java Iterator, File Object, Reentrant Lock, DNS Resolver, Raft Leader, Circuit Breaker, Rate Limiter, and Failover. These arise from protocols dominated by recursion, where SCC quotient collapses loops to single nodes, leaving a linear chain. A chain of n elements is trivially a bounded lattice.

Table 1 Benchmark results for 30 session type protocols. $|Q|$ = states, $|\delta|$ = transitions, $|SCC|$ = SCCs after quotient, $\|$ indicates use of parallel composition. All 30 form bounded lattices.

#	Protocol	$\|Q\|$	$\|\delta\|$	$\|SCC\|$	$\|$	Domain
1	Java Iterator	4	4	2		Pattern
2	File Object	5	5	4		I/O
3	SMTP	7	8	4		Network
4	HTTP Connection	5	7	3		Network
5	OAuth 2.0	7	11	6		Security
6	Two-Buyer	10	14	10	✓	Commerce
7	MCP	7	17	5	✓	AI Agent
8	A2A	5	7	4		AI Agent
9	File Channel	7	12	7	✓	I/O
10	ATM	7	11	6		Finance
11	Reentrant Lock	6	7	2		Concurrency
12	WebSocket	6	15	6	✓	Network
13	DB Transaction	5	9	3		Database
14	Pub/Sub	4	7	3		Messaging
15	DNS Resolver	4	8	2		Network
16	Reticulate Pipeline	14	18	14	✓	Tooling
17	GitHub CI Workflow	11	13	11	✓	CI/CD
18	TLS Handshake	13	15	13		Security
19	Raft Leader	6	11	2		Distributed
20	MQTT Client	8	20	6	✓	IoT
21	Circuit Breaker	6	10	2		Resilience
22	Connection Pool	13	29	5	✓	Database
23	gRPC BiDi Stream	5	9	5	✓	RPC
24	Blockchain Tx	6	8	6		Blockchain
25	Kafka Consumer	11	23	5	✓	Streaming
26	Rate Limiter	4	6	2		Resilience
27	Saga Orchestrator	20	37	20	✓	Distributed
28	Two-Phase Commit	7	8	7		Distributed
29	Leader Replication	13	24	3	✓	Distributed
30	Failover	6	8	3		Resilience

- *Trees/forests* (8 protocols): SMTP, HTTP Connection, A2A, ATM, DB Transaction, Pub/Sub, Blockchain Tx, TLS Handshake. These arise from protocols with branch/selection but no parallel. The resulting lattices are "wide" at branch points and "narrow" at convergence points.
- *Diamonds and wider lattices* (2 protocols): OAuth 2.0 and Two-Phase Commit. These have multiple incomparable states that share both a common upper and lower bound, creating diamond-like substructures.
- *Product lattices* (12 protocols): all protocols using $\|$. The product construction creates grid-like structures. The Connection Pool (13 states, 5 SCCs) is notable for combining two recursive branches in parallel, producing a product of two non-trivial quotient lattices.

Domain diversity.

The benchmarks span 12 application domains, demonstrating that session types (and their lattice structure) apply broadly. Of particular note are the AI agent protocols (MCP and A2A), which represent a new and rapidly evolving domain where formal verification of

interaction protocols is especially valuable. The MCP protocol uses parallel composition to model concurrent tool calls and notifications, while A2A uses nested recursion for the task lifecycle. Both form lattices, confirming that even novel protocol designs fall within the scope of Theorem 20.

Aggregate statistics.

Across all 30 benchmarks: the mean state count is 7.4 (median 6.5, range 4–20); the mean transition count is 12.3 (median 9, range 4–37); the mean SCC count after quotient is 5.8 (median 4.5, range 2–20). The ratio of SCCs to states (an indicator of how much recursion collapses the state space) ranges from 0.29 (Raft Leader: 6 states, 2 SCCs) to 1.0 (all non-recursive protocols, where every state is its own SCC).

7 Related Work

Session types.

Session types were introduced by Honda, Vasconcelos, and Kubo [14] for binary communication and extended to multiparty settings by Honda, Yoshida, and Carbone [15]. Vasconcelos [25] provides a comprehensive foundation. Session types for objects were developed by Dezani-Ciancaglini et al. [10], Gay et al. [12], and Vasconcelos et al. [26], with practical tool support from Mungo and StMungo [17, 18] and Scribble [27]. Gay and Hole [11] established the subtyping theory for session types.

Our work differs from this line in that we study the algebraic structure of *state spaces* rather than the types themselves. The subtyping lattice of session types is well-studied; we show that the individual state spaces are *also* lattices, a complementary and (to our knowledge) novel observation.

Lattices in programming languages.

Lattice theory appears prominently in abstract interpretation [6], where the abstract domain forms a lattice and the analysis computes a fixpoint. Our setting is different: the lattice is the state space itself, not an abstract domain over it. However, there is a conceptual connection: our Theorem 20 shows that session type state spaces are suitable domains for lattice-based analyses.

Type systems have long been connected to lattice theory [3, 9]; for instance, the type hierarchy of an object-oriented language often forms a lattice under subtyping. Our contribution is showing that the *behavioral* types (session types) produce lattices at the state-space level, not just at the type level.

Process algebra and concurrency.

The connection between session types and process calculi is well established [19, 21]. Dardha et al. [8] revisit this connection through linear logic, providing a Curry-Howard correspondence for session types. Scalas and Yoshida [22] show that multiparty session types can be decomposed into binary sessions while preserving safety and liveness. Our parallel constructor $\|$ differs from process-algebraic parallel composition in that it operates on a *single shared object* rather than on independent processes, requiring the product lattice construction of Definition 10.

Behavioral types.

Ancona et al. [1] survey the broader landscape of behavioral types, including session types, typestate, and contracts. The lattice structure we identify could potentially be applied to other behavioral type formalisms beyond session types. In particular, typestate systems [18] use state machines to track object protocols, and these state machines may also exhibit lattice structure under appropriate conditions.

Deadlock freedom and liveness.

Kobayashi [16] and Dardha and Gay [7] use type systems to ensure deadlock-freedom in concurrent processes. Our lattice structure provides a complementary perspective: the existence of meets and joins in the product lattice (Lemma 19) guarantees that concurrent execution paths always have well-defined synchronisation points. The termination condition in WF-Par (Corollary 21) ensures that these synchronisation points are eventually reached, which is related to (but distinct from) deadlock-freedom.

8 Conclusion

We have proved that the state space of every well-formed session type, after quotienting by strongly connected components, forms a bounded lattice (Theorem 20). This result, which we call the *Reticulate Theorem*, provides an algebraic foundation for session type analysis: branch and selection create meets and joins, recursion is absorbed by SCC quotient, and parallel composition yields product lattices. We validated the theorem on 30 diverse protocol benchmarks using two independently developed tools, and mechanically verified the key lemmas in Lean 4.

This lattice structure—the *reticulate*—opens several directions for future work.

Morphism hierarchy. A hierarchy of structure-preserving maps between reticulates—isomorphism, embedding, projection, and Galois connection—would enable protocol comparison and refinement checking. An embedding $\mathcal{L}(S_1)/{\equiv} \hookrightarrow \mathcal{L}(S_2)/{\equiv}$ would witness that S_1 is a "sub-protocol" of S_2, while a Galois connection would relate protocols at different levels of abstraction. We have implemented these in our tools [5] and are developing the theory.

Bisimulation connection. Bisimulation is the standard equivalence for labeled transition systems [19]. Since our state spaces are LTS with additional lattice structure, a natural question is how bisimulation relates to lattice isomorphism. We conjecture that bisimilar session types have isomorphic reticulates, but the converse may not hold (lattice isomorphism is coarser than bisimulation).

Abstract interpretation. The lattice structure of session type state spaces makes them natural candidates for abstract domains in the sense of Cousot and Cousot [6]. A Galois connection between a concrete program state space and the session type's reticulate would yield a sound abstract interpretation for protocol conformance checking.

We are pursuing these directions as part of the broader "Session Types as Algebraic Reticulates" research programme [4].

References

1 Davide Ancona, Viviana Bono, Mario Bravetti, Joana Campos, Giuseppe Castagna, Pierre-Malo Deniélou, Simon J. Gay, Nils Gesbert, Elena Giachino, Raymond Hu, et al. Behavioral types in programming languages. In *Foundations and Trends in Programming Languages*, volume 3, pages 95–230. Now Publishers, 2016. doi:10.1561/2500000031.

2 Anthropic. Model context protocol specification. Technical report, Anthropic, 2024. https://modelcontextprotocol.io/specification.

3 Garrett Birkhoff. *Lattice Theory*, volume 25 of *Colloquium Publications*. American Mathematical Society, 3rd edition, 1967.

4 Alexandre Zua Caldeira and Vasco T. Vasconcelos. The parallel constructor ‖ for session types on objects — formal specification (draft v0.2). Technical report, LASIGE, Faculdade de Ciências, Universidade de Lisboa, 2025. Technical report.

5 Alexandre Zua Caldeira and Vasco T. Vasconcelos. Reticulate: A lattice verification tool for session types with parallel composition. Technical report, LASIGE, Faculdade de Ciências, Universidade de Lisboa, 2025. Technical report (companion tool paper).

6 Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. pages 238–252, 1977. doi:10.1145/512950.512973.

7 Ornela Dardha and Simon J. Gay. A new linear logic for deadlock-free session-typed processes. In *Foundations of Software Science and Computation Structures (FoSSaCS)*, pages 91–109. Springer, 2018. doi:10.1007/978-3-319-89366-2_5.

8 Ornela Dardha, Elena Giachino, and Davide Sangiorgi. Session types revisited. *Information and Computation*, 256:253–286, 2017. doi:10.1016/j.ic.2017.06.002.

9 B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002. doi:10.1017/CBO9780511809088.

10 Mariangiola Dezani-Ciancaglini, Dimitris Mostrous, Nobuko Yoshida, and Sophia Drossopoulou. Session types for object-oriented languages. In *European Conference on Object-Oriented Programming (ECOOP)*, pages 328–352. Springer, 2006. doi:10.1007/11785477_20.

11 Simon J. Gay and Malcolm Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2–3):191–225, 2005. doi:10.1007/s00236-005-0177-z.

12 Simon J. Gay, Vasco T. Vasconcelos, António Ravara, Nils Gesbert, and Alexandre Zua Caldeira. Modular session types for distributed object-oriented programming. In *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 299–312. ACM, 2010. doi:10.1145/1706299.1706335.

13 Google. Agent-to-agent (A2A) protocol. Technical report, Google, 2024. https://google.github.io/A2A/.

14 Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *European Symposium on Programming (ESOP)*, volume 1381 of *Lecture Notes in Computer Science*, pages 122–138. Springer, 1998. doi:10.1007/BFb0053567.

15 Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 273–284. ACM, 2008. doi:10.1145/1328438.1328472.

16 Naoki Kobayashi. A new type system for deadlock-free processes. In *CONCUR*, pages 233–247. Springer, 2006. doi:10.1007/11817949_16.

17 Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. Typechecking protocols with Mungo and StMungo. In *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming (PPDP)*, pages 146–159. ACM, 2016. doi:10.1145/2967973.2968595.

18 Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. Typechecking protocols with Mungo and StMungo: A session type toolchain for Java. In *Behavioural Types: from Theory to Tools*, pages 371–395. River Publishers, 2017.

19 Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.

20 Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi:10.1007/978-3-030-79876-5_37.

21 Davide Sangiorgi and David Walker. *The π-Calculus: A Theory of Mobile Processes*. Cambridge University Press, 2001.

22 Alceste Scalas and Nobuko Yoshida. Less is more: Multiparty session types revisited. In *Proceedings of the ACM on Programming Languages (POPL)*, volume 3, pages 1–29. ACM, 2019. doi:10.1145/3290343.

23 Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972. doi:10.1137/0201010.

24 The mathlib Community. The Lean mathematical library. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020. doi:10.1145/3372885.3373824.

25 Vasco T. Vasconcelos. Fundamentals of session types. *Information and Computation*, 217:52–70, 2012. doi:10.1016/j.ic.2012.05.002.

26 Vasco T. Vasconcelos, Simon J. Gay, and António Ravara. Type checking a multithreaded functional language with session types. *Theoretical Computer Science*, 368(1–2):64–87, 2006. doi:10.1016/j.tcs.2006.06.028.

27 Nobuko Yoshida, Raymond Hu, Rumyana Neykova, and Nicholas Ng. The Scribble protocol language. In *Trustworthy Global Computing (TGC)*, pages 22–41. Springer, 2013. doi:10.1007/978-3-319-05119-2_3.